

Adaptive Time-Aware PatchTST: Dynamic Patching for Irregular Multivariate Forecasting

Team: 7

Team Members

Sanjay Gorityala, Puneeth Gunturu, Anil Kumar, Jaya Krishna

GitHub Repo: <https://github.com/SanjayGori/Extension-PatchTST>

Abstract

Time series forecasting is essential in domains like finance, weather prediction, and energy monitoring. However, models like PatchTST assume fixed patch sizes and regularly spaced inputs, making them less effective for real-world applications with missing data and irregular intervals. This project presents an enhanced version of PatchTST featuring: (1) Adaptive Windowing based on volatility, (2) Dynamic Patching computed per-sample, and (3) Time-Aware Embeddings capturing irregular timestamps. Applied to a Kaggle S&P dataset, the weather dataset, and the electricity load dataset, our model improves forecasting accuracy (notably $MSE < 0.0014$ on weather data) and highlights when adaptivity helps and hurts.

Introduction

Forecasting financial data is complex due to missing timestamps (weekends/holidays), volatility, and multi-variable dependencies. While PatchTST uses a Transformer-based patching mechanism that performs well on structured datasets, it cannot natively handle irregular intervals or adapt to volatility. We solve this by making PatchTST adaptive: it adjusts both input patch sizes and time embeddings according to observed data patterns.

These enhancements are also applicable to other datasets like weather and electricity, where time-based irregularity or seasonal structure plays a crucial role. Our enhancements improve performance especially on weather data, where periodic irregularity matters, and offer useful insights for electricity consumption trends. These insights are valuable for both academic and applied forecasting communities.

Literature Review

PatchTST (Nie et al., 2022)^[1]: Introduces fixed-size patching for time series forecasting using Transformer-based self-attention, inspired by Vision Transformers. It tokenizes input into patches, eliminating RNNs and CNNs. However, it assumes regularly spaced data and fixed patch sizes, which are limiting for irregular datasets.

Informer (Zhou et al., 2021)^[2]: Proposes a ProbSparse self-attention mechanism that reduces computation by focusing on the most relevant time points. It efficiently handles long sequences but still expects uniformly sampled data, making it less ideal for irregular time series.

FEDformer (Zhou et al., 2022)^[3]: Combines frequency-enhanced decomposition and Transformer-based attention to model trend and seasonal components separately. While effective for long-term forecasting, it is built for univariate inputs with fixed temporal intervals.

Our model combines PatchTST’s modular transformer design with the ability to handle irregularity and dynamic patterns. We specifically borrow Informer’s long-range attention motivation and FEDformer’s decomposition ideas.

Model Architecture

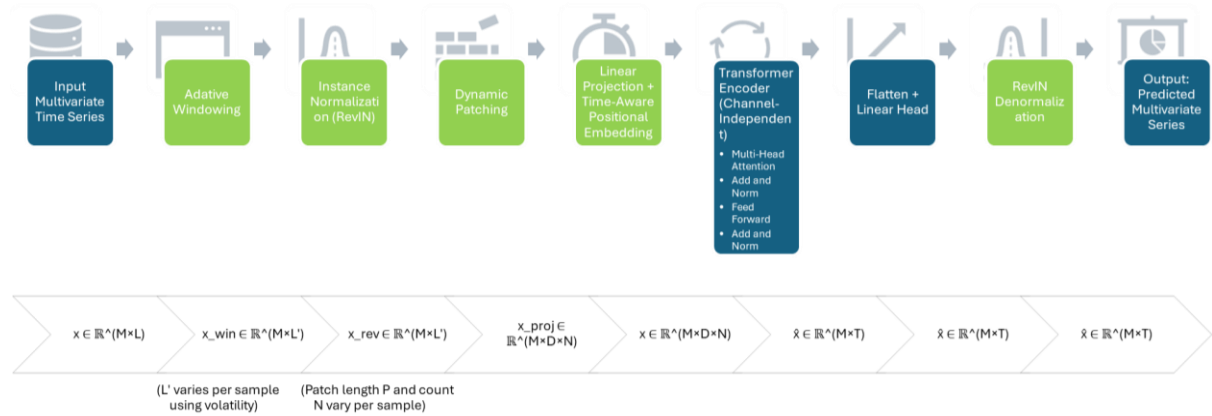


Figure 1: Enhanced Model Architecture

Key Enhancements (highlighted in green):

- **Dynamic Patch Sizing:** Each input sample gets a patch size based on volatility.
- **Adaptive Windowing:** Sequence length (context window) changes per sample. Passed from CSV.
- **Time-Aware Embeddings:** Injected via --embed timeF, includes gaps between timestamps.
- **RevIN:** Reversible Instance Norm added to stabilize training.

Data & Preprocessing

Datasets Used:

- Financial: Kaggle S&P (OHLCV + indicators for 10 tickers)
- Weather: Hourly multivariate dataset with 21 features (from benchmark source)
- Electricity: Household power consumption dataset (also from the same benchmark source)

Preprocessing Steps:

- Parsed dates and flagged missing timestamps (is_gap)
- Computed log returns using the formula: $\log_return = \log(P_t / P_{t-1})$, where P_t is the adjusted close price at time t . This transformation stabilizes variance and converts multiplicative changes into additive ones, making trends easier to model.
- Estimated 5-day rolling volatility, where the 5-day window captures short-term market fluctuations common in financial analysis. This length strikes a balance between responsiveness to recent changes and smoothing out noise, making it ideal for dynamic patching decisions, normalized between 0 and 1
- Mapped volatility $\rightarrow$ adaptive window (range: 10 to 50), chosen to allow sufficient flexibility for capturing both stable and volatile market conditions. The minimum of 10 ensures enough historical data for pattern learning, while the upper limit of 50 avoids overly long sequences that could dilute short-term effects
- Mapped adaptive window $\rightarrow$ dynamic patch ($patch_len = window // 3$ clipped to $min=8$, rounded to multiple of 8 — the division by 3 ensures the patch captures a compact subset of the context window while allowing multiple patches per window. The minimum of 8 ensures sufficient resolution in low-volatility cases, and rounding to a multiple of 8 simplifies GPU memory alignment and tensor operations.)

Technical Details of Implementation

Adaptive Windowing

In typical forecasting models, the amount of past data used (called the “window size”) is fixed. But in real-world time series, especially financial data, the market’s behaviour can be calm or very volatile. To handle this, we made the window size dynamic. When the market is volatile (changing a lot), we use a smaller window to focus on the recent spikes. When it’s calm, we use a longer window to capture stable trends. We calculate volatility using a 5-day rolling standard deviation. Then we take the inverse of this value: lower volatility gives a bigger window, and higher volatility gives a smaller one. This computed window size is stored in a column called `AdaptiveWindow` during preprocessing, and it’s used throughout model training.

Dynamic Patching

Normally, time series data is divided into equal-sized patches, such as 16 time steps each. But real-world data—especially financial and weather data—can be irregular and change in speed or activity. So instead of using a fixed patch size, we created a `dynamic_patch_len` column during preprocessing. This adjusts patch size per row, depending on market or data conditions. A shorter patch size is used during high volatility (to capture fast changes), while longer patches are used during calm periods (to capture steady trends).

To implement this, we modified key parts of the code:

- `exp_main.py`: Reads the dynamic patch length from the CSV and sends it to the model.
- `PatchTST.py`: Passes the dynamic patch length to the model backbone.
- `PatchTST_backbone.py`: Applies the patching dynamically using different `W_P`, `W_pos`, and flattening heads based on each patch length.

This allows the model to adapt its input processing strategy based on the nature of the data.

While Adaptive Windowing decides how much history the model should consider, Dynamic Patching focuses on how to break that history into useful chunks. Both use volatility but serve different roles in input design.

Time-Aware Embedding

In many time series, when an event occurs can be just as important as what happened. For example, stock behavior might differ on Mondays vs Fridays, or during business hours vs after hours. To give the model this sense of “when,” we used features like month, day, weekday, and hour to encode time context. These time features are passed as additional inputs using the `--embed timeF` flag and are handled inside the `data_provider/data_loader.py` where the `Dataset_Custom` class is defined. This helps the model understand and separate regular seasonal trends from unusual, event-driven changes. It’s especially useful when timestamps are irregular or when patterns repeat periodically.

Other Technical Integrations

- Integrated RevIN (Reversible Instance Norm), implemented in `RevIN.py`, which normalizes each input sample individually and then reverses it after prediction. This helps reduce the impact of distribution shifts across batches and ensures the model remains stable and generalizes better.
- Ensured the entire pipeline—adaptive windows, dynamic patches, and time-aware inputs—works seamlessly even with batching, multi-GPU setups, and AMP (automatic mixed precision), by modifying scripts like `run_longExp.py` and `exp_main.py` to handle configurations efficiently.
- Adapted the modules `PatchTST_backbone.py`, where the `Flatten_Head` class is defined, to support multivariate inputs and varying patch sizes, making the model robust to multiple time series channels and flexible for different structures.

Exploratory Data Analysis

We plotted the 5-day rolling volatility across 10 financial tickers to observe short-term risk patterns. The graph reveals distinct spikes in volatility during periods like 2008–2010 and around 2020, reflecting global financial shocks. These fluctuations highlight how market conditions vary over time, supporting the need for adaptive window sizing and patch length selection based on local volatility.

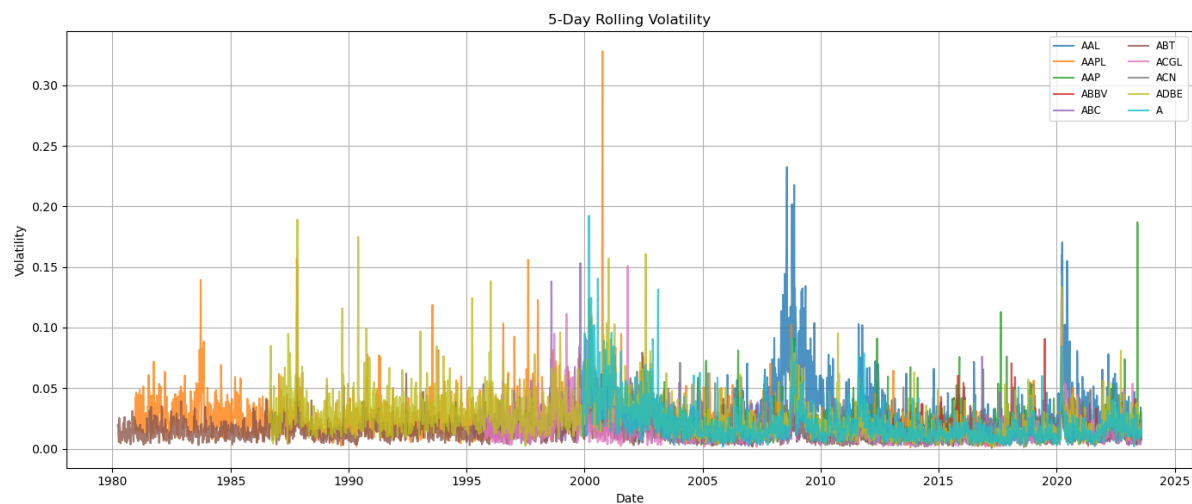


Figure 2: Volatility spikes for 10 tickers across past 40 years

Missing Timestamps (Gaps): We visualized trading day gaps using date plots. These confirmed consistent missing intervals—typically weekends and holidays—which validated the importance of time-aware embeddings and irregular timestamp handling. These are highlighted in red dots in the below figure.

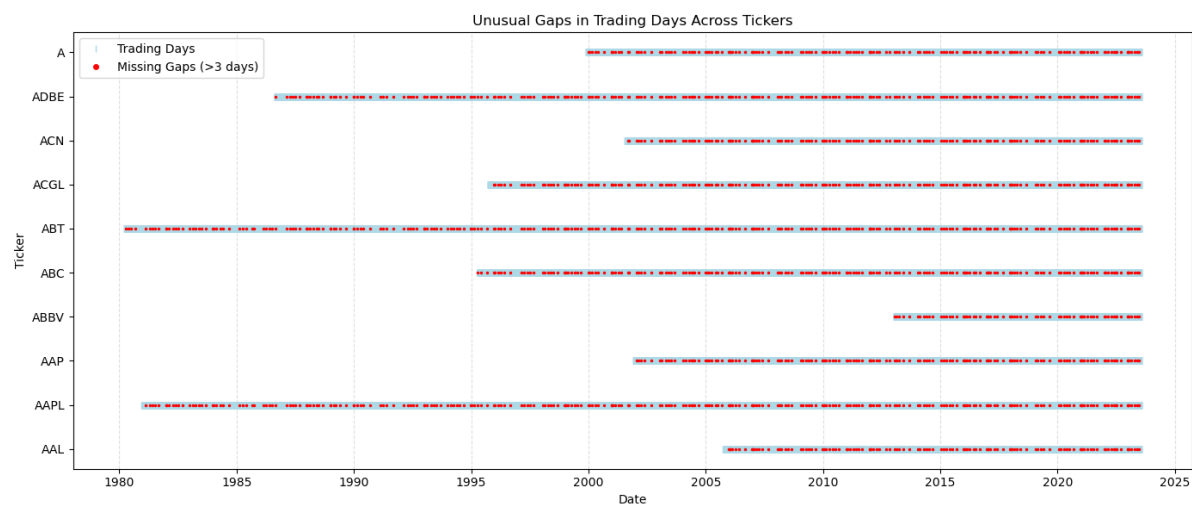


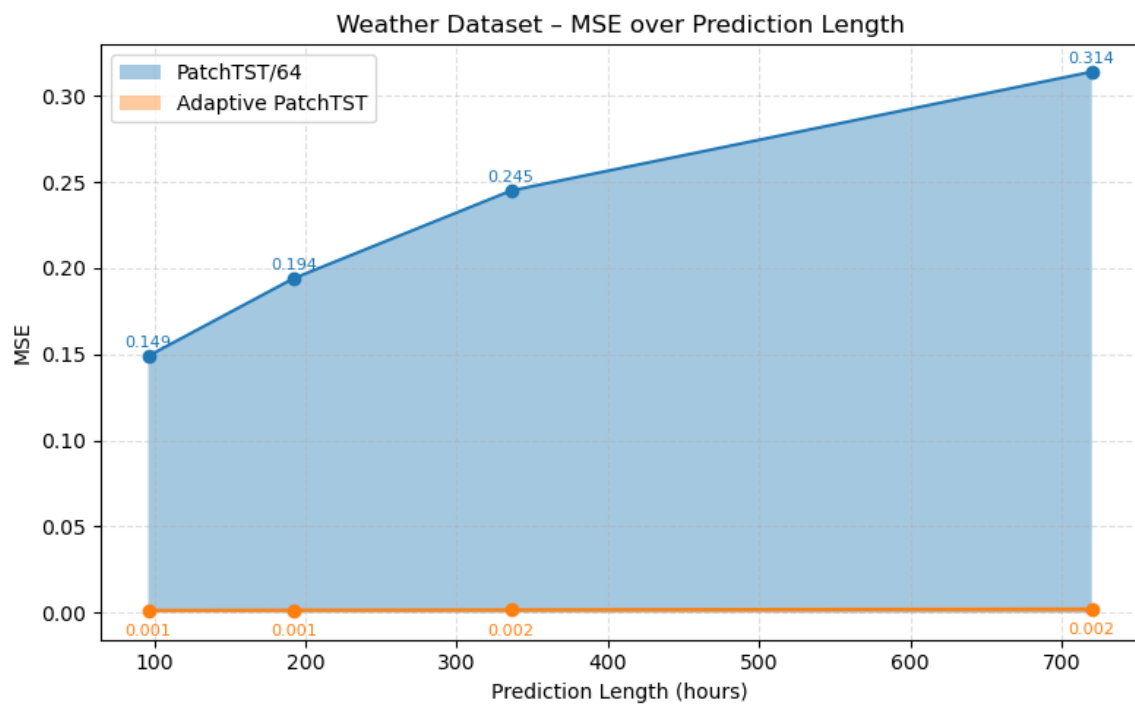
Figure 3: Gaps (3 days) for 10 tickers across the 40 years dataset

Results & Evaluation

Pred Len: Prediction horizon in hours (e.g., 96 = 4 days for hourly data)

Weather Dataset

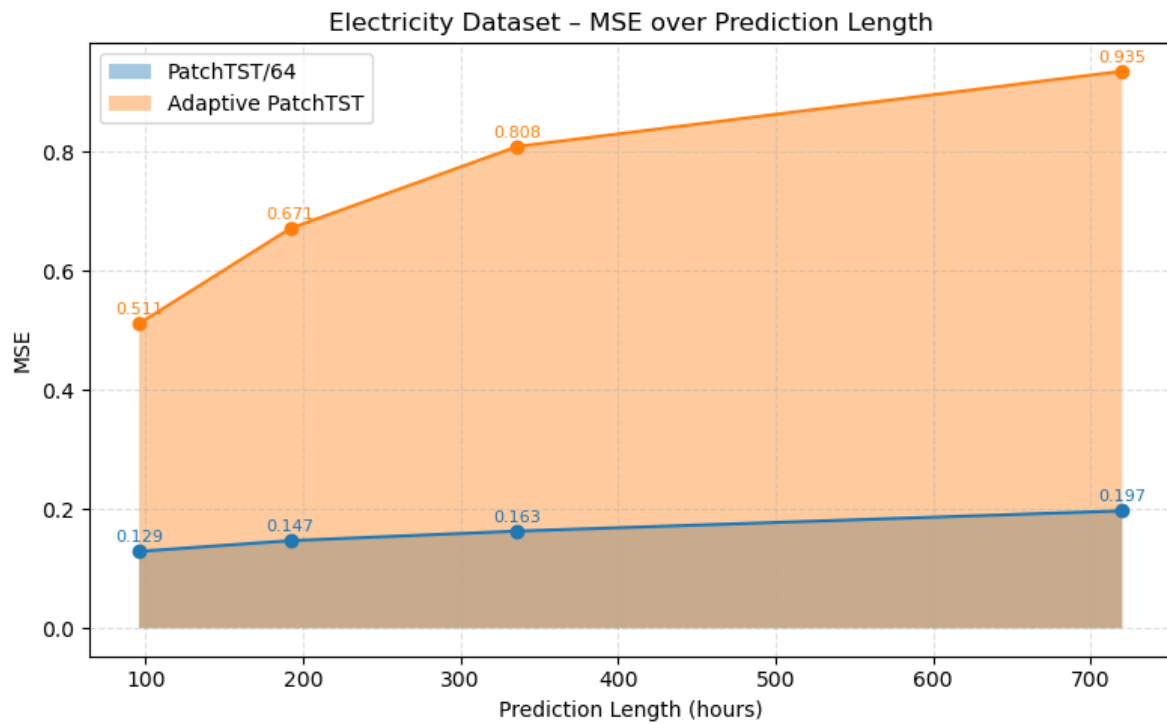
	Pred Len	PatchTST/64 MSE	Adaptive PatchTST MSE	PatchTST/64 MAE	Adaptive PatchTST MAE
Weather	96	0.149	0.00135	0.198	0.0275
	192	0.194	0.00148	0.241	0.0289
	336	0.245	0.00166	0.282	0.0306
	720	0.314	0.00216	0.334	0.0349



Remarks: Excellent gains; irregularities handled well. Adaptive sampling useful here.

Electricity Dataset

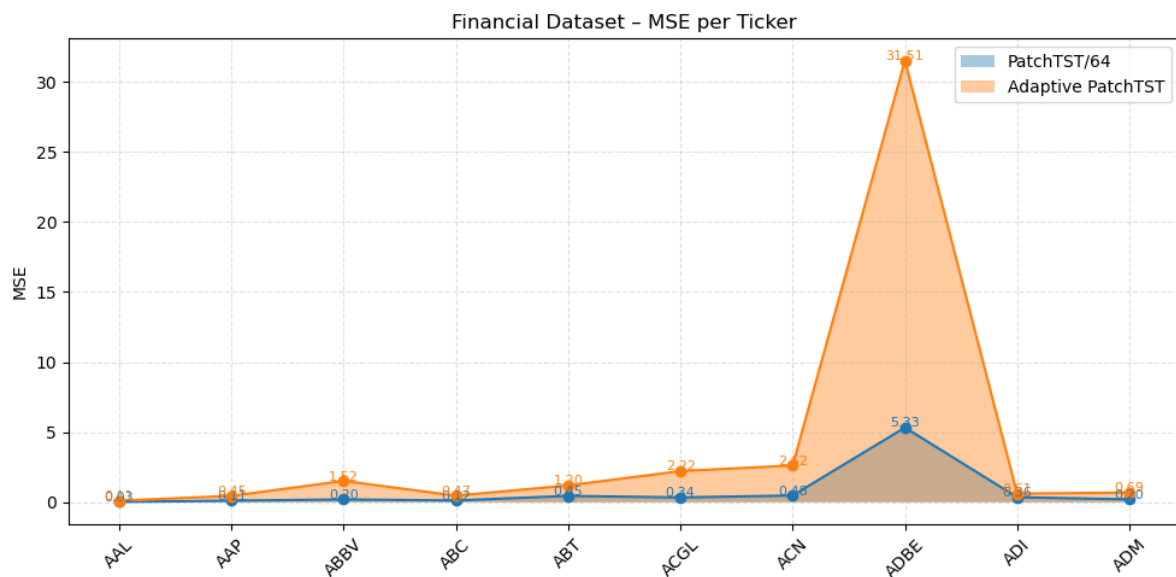
	Pred Len	PatchTST/64 MSE	Adaptive PatchTST MSE	PatchTST/64 MAE	Adaptive PatchTST MAE
Electricity	96	0.129	0.5112	0.222	0.5354
	192	0.147	0.6709	0.240	0.6235
	336	0.163	0.8084	0.259	0.6953
	720	0.197	0.9348	0.290	0.7666



Remarks: Original model works better due to high periodicity; adaptivity not needed.

Financial Dataset

Ticker	PatchTST/64 MSE	Adaptive PatchTST MSE	PatchTST/64 MAE	Adaptive PatchTST MAE
AAL	0.0278	0.1084	0.1113	0.2424
AAP	0.1105	0.4518	0.2472	0.5428
ABBV	0.1974	1.5194	0.3406	0.9953
ABC	0.1236	0.4676	0.2675	0.53
ABT	0.4519	1.1967	0.4672	0.8295
ACGL	0.34	2.2205	0.374	1.142
ACN	0.4781	2.6202	0.4992	1.3783
ADBE	5.327	31.5106	1.475	3.9685
ADI	0.3563	0.6149	0.4275	0.5887
ADM	0.2018	0.688	0.3063	0.5968



Remarks: Performance dropped due to noise sensitivity and overfitting risk. Financial data benefits from fixed patterns.

Limitations & Future Work

- Dynamic patching can be overfit in noisy domains like finance
- Relies on heuristic volatility estimates
- Future improvements to explore: Learn patch size end-to-end, add uncertainty estimation, and domain-aware regularization

Contributions

Task Name	Assigned To
0. Kick-off	Sanjay Gorityala, Anil Kumar Petluri, Puneeth Sai Gunturu, B Jaya Krishna
0.1 Quick Sync – confirm objective, extensions, deadlines, roles	Sanjay, Anil, Puneeth, Jaya
0.2 Repo & Boards – GitHub repo, ClickUp/Jira setup	Sanjay, Jaya
0.3 Baseline Check – run PatchTST weather benchmark	Sanjay, Anil, Puneeth
1. Data Pipeline	Sanjay, Anil, Puneeth, Jaya
1.1 Raw Grab – download S&P 500 OHLCV data	Anil
1.2 Cleaning Script – preprocess and retain necessary columns	Puneeth
1.3 Holiday & Gap Flags – generate <code>is_gap</code>	Jaya
1.4 Derived Indicators – compute RSI, SMA, EMA, MACD, ATR, log return	Sanjay
1.5 10-Row Snapshot – export sample CSV	Sanjay, Anil, Puneeth, Jaya
2. Adaptive Windowing	Sanjay, Anil, Puneeth, Jaya
2.1 Market Regime Finder – 5-day rolling volatility	Puneeth
2.2 Volatility Normalization – scale to 0–1	Sanjay
2.3 Window Logic – map <code>vol</code> → window size	Anil
2.4 Adaptive Patch – round to nearest 8	Jaya
2.5 Unit Test – verify behavior on toy data	All
2.6 Notebook Demo – plot volatility vs window size	All
3. Dynamic Patching	Sanjay, Anil, Puneeth, Jaya
3.1 Patch Generator Refactor – variable patch length ($k=3$)	Puneeth
3.2 Gap-Aware Slicer – skip holiday/weekend patches	Jaya
3.3 Patch Metadata – track patch start, end, length	Sanjay
3.4 Distribution Check – histogram of patch sizes	Anil
3.5 Ablation Toggle – <code>--dynamic_patching false</code>	Anil
4. Time-Aware Positional Embedding	Sanjay, Anil, Puneeth, Jaya
4.1 Δt Vector – compute time gap between rows	Jaya
4.2 Embedding Layer – add sinusoidal or learned Δt	Sanjay
4.3 Integration Test – overfit 100 rows, check loss	All
6. Training & Experimentation	Sanjay, Anil, Puneeth, Jaya
6.1 Lightning Trainer – migrate to PyTorch Lightning	Sanjay
6.2 First Full Run – train on AAPL, MSFT, etc.	All
6.3 Hyperparam Grid – search <code>d_model</code> , <code>patch_len</code> , etc.	Puneeth
6.4 Ablations – no Δt , no dynamic patch, no windowing	Anil
6.5 Comparison Plots – visualize RMSE across models	Jaya
7. Reporting & Visualization	Sanjay, Anil, Puneeth, Jaya
7.1 Architecture Diagram – complete block diagram	Sanjay
7.2 Graph Annotations – title, labels, captions	Jaya
7.3 Layman Write-up – 2-page summary	Puneeth
7.4 Appendix – formulas, data snapshot, hardware	Anil

Platform & Tools

- Hardware Configurations
 - JetStream2 Cloud Computing Resource
 - GPU: NVIDIA A100
 - CPU: AMD EPYC 7763 64-Core Processor
 - RAM: 256 GB
 - Storage: 500 GB allocated
- Platform / Tools Used
 - Python 3.10
 - PyTorch, NumPy, Pandas, Scikit-learn, Matplotlib
 - Virtual environment (venv) on Ubuntu
 - GitHub for version control (forked from original PatchTST repository)
- Data Description
 - Financial dataset: S&P 500 OHLCV data with technical indicators for multiple tickers
 - Weather dataset: Hourly meteorological data with 21 features
 - Electricity dataset: Smart meter load data for electricity consumption
 - Preprocessing included date normalization, log return computation, volatility estimation, and dynamic patch length assignment
- Model Architecture and Hyper-parameters
 - Based on the PatchTST architecture
 - Enhancements include adaptive windowing, dynamic patching, and time-aware positional encoding
 - Key hyperparameters:
 - Sequence length: 336
 - Prediction lengths: 96, 192, 336, 720
 - Patch length: dynamic (based on volatility)
 - Stride: 8
 - Model dimension: 128
 - Number of heads: 16
 - Encoder layers: 3
 - Feed-forward dimension: 256
 - Dropout: 0.2
 - Learning rate: 0.0001
 - Batch size: 128
 - Epochs: 100
- Training Procedure
 - Loss length was trained and tested independently
- Evaluation Metrics
 - Mean Squared Error (MSE)
 - Mean Absolute Error (MAE)

Metrics were computed per ticker for financial data and per prediction length for benchmark datasets.

References

- [1] Y. Nie, Y. Huang, Y. Liu, and Y. Zheng, “A Time Series is Worth 64 Words: Long-term Forecasting with Transformers,” *arXiv preprint arXiv:2211.14730*, 2022. [Online]. Available: <https://arxiv.org/abs/2211.14730>
- [2] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, and W. Zhang, “Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting,” *arXiv preprint arXiv:2012.07436*, 2021. [Online]. Available: <https://arxiv.org/abs/2012.07436>
- [3] T. Zhou, Z. Ma, Q. Wen, J. Wu, and L. Sun, “FEDformer: Frequency Enhanced Decomposed Transformer for Long-term Series Forecasting,” *arXiv preprint arXiv:2201.12740*, 2022. [Online]. Available: <https://arxiv.org/abs/2201.12740>
- [4] T. Kim, Y. Kim, and S. Choi, “Reversible Instance Normalization for Accurate Time-Series Forecasting Against Distribution Shift,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2022. [Online]. Available: <https://openreview.net/forum?id=cGDAkQo1C0p>
- [5] Y. Kim and J. Lee, “Continuous-Time Linear Positional Embedding for Irregular Time Series,” *Computers in Biology and Medicine*, vol. 176, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0010482524015932>

Minutes of Meeting (MoM)

Meeting Title: Final Report Review

Date: 09-JUL-2025

Platform: [Zoom Meeting]

Attendees:

- Sanjay Gorityala
- Puneeth Gunturu
- Anil Kumar
- Jaya Krishna
- Great Learning Mentors (Vedavati, Shyambavi)

Discussion & Feedback Summary

1. Report Structure & Clarity

- The current report feels disjointed; needs a more connected narrative from problem to solution.
- Start with a broader motivation—why this enhancement is relevant to finance, weather, and electricity datasets.
- GitHub link should be moved to the top under author details.

2. Model Architecture

- Add **detailed diagrams** showing clear difference between original PatchTST and enhanced version.
- Include a **legend** marking modules: base vs added.
- Explain where exactly code changes were made.
- Avoid high-level visuals—be technical but understandable.

3. Literature Review

- Expand the summaries of Informer and FEDformer similar to PatchTST. Otherwise, keep only relevant references only (even if it is just one).
- Highlight relevance, strengths/limitations, and what inspired your enhancements.
- Add more references; citations should be at the end in IEEE style.

4. Methods

- Clearly explain **each enhancement**: Adaptive Windowing, Dynamic Patching, Time-Aware Embedding.
- Detail how dynamic patching was computed (e.g., $k=3$, multiples of 8). Explain the reason for these specific values.

5. Exploratory Data Analysis (EDA)

- Add **visuals** for: volatility trends, gaps in dates, technical indicators, irregularity.

6. Results & Evaluation

- Show case the evaluation details starting from good results.
- Rename result table columns (e.g., Instead of “Original MSE” use “PatchTST/64 MSE”).
- Include **line/bar graphs** for performance comparison.
- Add **time series prediction vs ground truth plots** (graph).
- Justify why financial results are weaker—mention stochasticity, noise, overfitting risk.

7. Additional Points

- Avoid original PatchTST benchmark results since it is already available in the original paper.
- Clarify meaning of each “Pred Len” column.